SymQ: Exact ANF Inspection and Equivalence Checking for Reversible Boolean Circuits

Task-separated qualification of a specialized native engine

Joshua Melka
Independent researcher (no affiliation)

Research revision, 30 October 2026 — not submitted

Abstract

We study an engineering trade-off in compiler verification: obtaining exact Boolean output functions and replayable counterexamples can cost more than deciding equivalence alone. SymQ represents algebraic normal forms (ANFs) as monomial zero-suppressed decision diagrams, a representation established by PolyBoRi and earlier functional-diagram work. Its contribution is a specialized budgeted implementation and inspectable experimental evidence, rather than a new algebraic algorithm. We separate simulation, explicit ANF materialization, equivalence, counterexample production and full benchmark service-call cost; SAT and quantum decision procedures are compared only where their semantic service matches. A source- and binary-pinned short qualification records 4320 planned trials over 8 cases in 4 families; 2700 trials on 5 cases in 4 families were attempted, with failures, unsupported requests and unrun cases retained. It validates a protocol on that bounded corpus, not broad competitiveness. Earlier measurements are preserved as historical, including adverse families, resource failures and corrected timing boundaries; they are not relabeled as current results. Monomial expansion and variable order remain important limitations. We claim neither a new representation, universal speedup, independently certified equivalence nor general quantum-program verification.

1 A compiler-facing question and contribution

Reversible X, CX, CCX and MCX circuits occur in arithmetic, oracles and uncomputation [24, 10]. Two equal-width programs in this fragment compute permutations of the Boolean cube, so their full-input equivalence is a classical functional question. A compiler may ask only whether a rewrite preserves that function, or may also need explicit intermediate ANFs, dependency inspection and a sparse counterexample. These services have different costs.

Our testable claim is narrow: a specialized native monomial-ZDD implementation can share exact symbolic Boolean functions and replay distinguishing inputs, while its practical position relative to explicit ANF, BDDs, PolyBoRi and SAT depends on the requested service, family, order and budget. We supply (i) an explicit semantic contract and short derivations, (ii) task-separated source-pinned qualification with raw status denominators, and (iii) preserved historical evidence, including unfavorable results. These are implementation and measurement contributions. The research implementation is not formally verified.

The historical paper branch used an older native binary and an explored corpus formerly called blind. Current code has ABI-5 validation, bounded caches and explicit wider modes; old timings cannot validate those changes. We therefore distinguish current qualification from historical exploration throughout. A short successful run can qualify a harness and its tested cases; it cannot establish machine-independent or cross-family superiority.

2 Known foundations and relevant competitors

Boolean ANFs are canonical square-free polynomial expansions. ZDDs [15, 16, 17] compactly represent set families; monomial families with recursive Boolean-ring operations are central to PolyBoRi/BRiAl [3, 4, 22]. Positive-Davio functional diagrams describe the same expansion under a fixed order [13, 9]. We claim no first representation, multiplication recurrence or Boolean verification theorem.

CUDD BDDs [5, 21, 8] represent functions without expanding ANF; their service may therefore be preferable for a verdict. DD and SAT checking of reversible circuits already appear in Wille et al. [23]. ABC combines AIG techniques with SAT-based combinational equivalence [2, 18]. A PySAT miter, when measured, is a SAT baseline rather than a claim to implement ABC. A solver need not export monomials to decide the same Boolean miter.

MQT QCEC [6, 7, 19] can establish unitary relations on a broader grammar. On the common reversible fragment, permutation-matrix equality agrees with full functional equality, but QCEC supplies no output ANFs. Its phase convention and enabled complete checkers must be recorded; inconclusive outcomes are not differences. Path sums [1], QuPRS [11] and hybrid methods address broader problems. Irene preserves factored XOR–AND graphs and reasons with hybrid path sums and observable density kernels [12]; phased outcome-complete simulation tracks phases under declared rotation/outcome correspondences [14]; deferred-measurement techniques lift unitary checking to hybrid programs [20]. They are important prior art, not unmatched speed comparisons. `paper/RELATED_WORK.md` records primary-source checks and remaining search limitations.

3 Contract and derivations

A well-formed width-n circuit is a finite sequence (C, t) with distinct controls $C \subseteq \{0, \ldots, n-1\}$ and $t \notin C$. Its gate updates $s_t \leftarrow s_t \oplus \prod_{c \in C} s_c$, leaving other wires fixed; the empty product is one. All n inputs are independent and all outputs are compared in the declared order. No clean ancilla, discarded output or legal-input predicate is inferred from circuit names.

Let $B_n = \mathrm{GF}(2)[x_0, \ldots, x_{n-1}]/(x_i^2 - x_i)$. Its unique ANF consists of distinct square-free monomials; evaluation is a ring homomorphism to GF(2) and identifies B_n with Boolean functions. Symbolic simulation starts at $f_i = x_i$ and updates $f_t \leftarrow f_t + \prod_{c \in C} f_c$.

Proposition 1 (Classical correctness, standard)**.** *For every input a, evaluating the symbolic outputs gives the concrete circuit output. Consequently, equal-width circuits are functionally equivalent exactly when all output ANFs agree.*

Proof. Induct over gates. Initially $f_i(a) = a_i$. Evaluation preserves addition and multiplication, so the symbolic target update evaluates to precisely the concrete XOR/product update. Uniqueness of ANF then gives both directions of the equivalence criterion. □

Proposition 2 (Minimum-degree distinguishing input)**.** *If nonzero $p \in B_n$ contains a monomial M of minimum cardinality, the assignment χ_M with ones exactly on M satisfies $p(\chi_M) = 1$.*

Proof. The evaluation counts modulo two the monomials of p contained in M. A proper subset would have smaller degree and cannot occur, so exactly M contributes. The argument also covers the constant monomial $M = \emptyset$. □

Applied to a differing output, this gives a concrete counterexample. Both circuits are re-executed before accepting it. That replay validates a difference; it does not certify an equivalence verdict or independently verify the implementation.

Fix a variable order. A ZDD node denotes $P_0 + vP_1$, with ordered children, nonzero high child and unique hash-consed triples. Root identity denotes equality only within the same manager and generation. Cross-manager identifiers must never be compared directly. For $P = P_0 + vP_1$ and $Q = Q_0 + vQ_1$,

$$PQ = P_0Q_0 + v(P_0Q_1 + P_1Q_0 + P_1Q_1). \tag{1}$$

Expansion and $v^2 = v$ prove this familiar recursion. The alternative form is $P_0Q_0 + v((P_0+P_1)(Q_0+Q_1)+P_0Q_0)$; its low term is essential. No asymptotic advantage is asserted for either form.

Budgets may abort but may not approximate. Under exact transitions, a completed in-budget run has the unbudgeted semantics; a late or aborted run establishes neither equality nor difference. This abstract observation does not prove responsiveness, safe cache recovery or C memory correctness. Those require separate implementation invariants and tests. The full model in `paper/FORMAL_MODEL.md` states manager ownership, root preservation, resource and recovery hypotheses explicitly. None of these proofs is machine-checked.

4 Implementation and task boundaries

The current native implementation retains the ordinary 64-variable engine and an explicit 128-variable engine, with multiword import/export and high-bit witnesses. Width extension changes admissibility, not exponential worst-case representation growth. Root sharing, batched gate propagation and owned DAG transfer are engineering mechanisms, not new mathematical decision procedures. Caches, live roots and temporary transfer peaks consume resources even when the final function is small. No backend is silently selected to rescue a failed declared engine. The primary `zdd-native-fused` configuration uses the shipped public native gate loop; the secondary `zdd-native` benchmark configuration explicitly substitutes a matched Python algebra loop. It is not the ordinary default, and the two configurations must not be selected post hoc or pooled.

Table 1: Services and comparison boundaries. Conversion and replay are included in the full benchmark service cost when required. Actual public API calls, manual benchmark services and conversion-excluded services retain distinct raw labels; no service is called kernel-only.

service	required output	eligible comparison
Simulation	exact symbolic Boolean functions	ring/BDD engines; no monomial enumeration required
ANF export	explicit ANF of every retained wire	ring engines and a separately timed BDD-to-ANF conversion
Equivalence	exact full-input Boolean decision	symbolic engines, checked SAT miter; QCEC on common unitary grammar
Counterexample	distinguishing assignment, concretely replayed	only adapters actually returning a witness; verdict-only negatives do not qualify

A native benchmark first validates the actual ABI, result shape, root count, loaded extension and source and build fingerprints. Replacing an obsolete ABI assertion alone would not establish compatibility. The harness exercises simulation and both verdict directions against an independent small reference. No ABI-4 fallback or stale binary is retained to make tests pass.

5 Current short qualification protocol

The campaign is frozen before execution by its manifest, source commit, per-file hashes, native binary hash and environment. Repetition order is randomized in paired blocks; workers are fresh and resource-limited, with CPU, wall and memory boundaries recorded. Teardown contributes to whole-worker lifetime; an exclusive cleanup duration is not instrumented. The preregistered study combines newly seeded synthetic cases with a deterministic increment calibration; it is not an entirely untouched holdout. Whole-worker wall time extends through process reaping. Raw `cpu_seconds` is cumulative CPU sampled before result JSON encoding and teardown; those final CPU costs are unmeasured. `service_cpu_seconds` runs through the service checkpoint, including input materialization and orchestration, and therefore has a broader boundary than the selected public-call `service_seconds`. Validation CPU is separate. Workers eagerly import the same optional-tool prefix before their service-ready checkpoint. Its startup cost and memory remain in whole-worker measurements, outside service time. Consequently, RSS here characterizes this qualification process envelope, not the memory of a minimal deployment importing only its chosen backend or an isolated native heap. The executed qualification subset is now explored; reserved unrun cases remain unmeasured, not independently gathered external workloads. During execution, code inspection found that the increment generator ignores its seed and its small-width instance had already been used in development tests. Its raw `new_holdout` label is therefore incorrect: an adjudication sidecar classifies it as calibration while preserving the measured source, manifest and raw bytes. No case was replaced or rerun. The original aggregate remains a descriptive mixed-study result; a separately labeled post-audit holdout-only sensitivity excludes calibration, leaving only two small oracle-eligible families. Its provenance is synthetic unless a row identifies a pinned external generator and license; a fresh seed is not evidence of external-workload generality.

The affine pair uses a random CNOT stream and independent Gaussian synthesis. The triangular shear keeps its control wires fixed and compares ANF coefficients with a truth-table-minterm implementation on the last target. Increment compares a carry construction with complement/decrement/complement. The adverse nonlinear pairs instead copy one random MCX stream and insert an adjacent cancelling CX pair into the candidate. They stress propagation through a connected synthetic circuit; they are not independent compiler outputs, and structural cancellation can avoid the symbolic work entirely.

The raw label `public_api_end_to_end` denotes the actual public API path named in the artifact, loaded from the immutable source snapshot with dependencies installed in the recorded environment. This is a source-snapshot performance campaign, not a wheel performance campaign; independent wheel CI checks packaging separately. The frozen manifest's older word "installed" does not change this loading boundary. An inclusive API stage has no invented internal decomposition. The separate `task_service_end_to_end` label denotes the manual benchmark pipeline with input materialization, engine setup and required conversion/replay. A `precompiled_service` row excludes conversion but still includes setup; it is not a kernel-only time. Independent oracle validation is separately timed outside service cost under its own validation wall/CPU budget, within the complete worker envelope. The service checkpoint survives optional validation failure: a fast completed service is not relabeled a service timeout because its independent oracle later exhausts resources. Qualified success, service completion and oracle/replay evidence therefore have distinct counts.

The raw artifact retains all requested trials: success, unsupported, not run, timeout, OOM, inconclusive and error. A larger unrun instance is not counted as a measured timeout. Computation, comparison, witness

extraction/replay, export and validation have separate stage fields; unavailable stages remain unavailable. A completed simulation is not automatically a completed export. Likewise, a fingerprint singleton supplies no cross-backend agreement. The declared analysis plan fixes the shipped fused native configuration as primary and retains the matched Python-loop native configuration as a separate ablation. Small all-input oracles and concretely checked mutants are correctness evidence distinct from timing and from solver decisions.

Current conditional speed compares paired successful trials with independent small-domain oracle checks for the same case, task and boundary. Larger cross-algorithm agreements are separate evidence and do not silently acquire exhaustive-oracle status. We take a median of paired ratios within each case, then give each case within a family and each represented family equal weight. Descriptive resampling intervals and leave-one-family-out sensitivity expose dependence and influence; a few synthetic families are not thirty independent observations. No population-level generalization follows from this small set. Coverage is separately reported under the joint declared policy: common process wall, CPU and memory limits plus backend-specific cooperative representation guards. The term threshold is 200000 and the node threshold 1000000, but these are not equal abstract-work budgets. Sparse ANF checks explicit term counts and an input-product-size bound; PolyBoRi checks polynomial term counts. Python/native ZDDs limit retained manager nodes, while their compact functions need not satisfy the term threshold until ANF materialization. CUDD limits manager nodes and applies the term guard during conversion to ANF. SAT and QCEC receive their applicable timeout/process limits, without artificial ANF-term or DD-node caps. Thus coverage is not a comparison under CPU/RSS limits alone, and an early representation-guard refusal must not be interpreted as exhaustion of a uniform amount of work. Parent resource polling occurs at 10 ms intervals; process termination and cleanup have latency. A late result is withdrawn, and elapsed process lifetime is not promised to stop at the nominal service limit with zero overshoot. Controlled timeouts forcibly terminate workers; a zero `killed` outcome count means no unexplained kills, not that no operating-system termination occurred.

6 Current qualification evidence

The current artifact has 4320 planned trials, 8 cases and 4 families. Only 2700 trials on 5 cases in 4 families were attempted; the remaining planned cases remain explicit `not_run` records rather than measured timeouts. Its full source commit and binary hash are in the artifact identity below. Tables 2 and 5 are generated from unmodified raw rows by an independently recomputing paper script. The script refuses duplicate trial keys, unpinned completed rows, successful ANF exports without materialization, and successful counterexamples without concrete replay.

Observed status and evidence. The raw records contain 240 inconclusive, 1620 not_run, 1560 ok, 525 timeout, 375 unsupported. Among successful trials, 1395 record an independent small-domain oracle check and 405 record concrete witness replay. These are different evidence counts, not additional independent programs. Unrun cases provide no measured representation frontier, and unsupported requests do not establish slowness. Actual failures on attempted cases remain measured evidence under their stated limits.

Conditional family effects. Ratios below concern only the public API boundary and common successful, independently oracle-checked paired repetitions. Native means the predeclared shipped `zdd-native-fused` public configuration, not a post-hoc best native mode. Each family receives one aggregate vote. Intervals are descriptive cluster resampling, not justified cross-family population confidence. These original aggregates describe the mixed study, including deterministic increment calibration, rather than a pure untouched holdout. Table 7 separately reports the post-audit holdout-only sensitivity.

Absolute cost and boundary. Median public equivalence service time over oracle-checked repeat rows is zdd-native-fused: 0.129 ms; zdd: 0.259 ms; polybori: 35.834 ms; cudd-bdd: 11.579 ms; sat: 1.014 ms. These are tiny programs and complete configured API calls including per-call manager construction, with process startup/imports excluded. Their large ratios against optional packages do not isolate a C-kernel advantage or establish larger-workload competitiveness. Table 3 shows the distinct adverse decision/witness frontier; the native node guard is an observed manager-allocation frontier under the declared policy, not a final-DAG lower bound or proof that different orders or collection cannot help.

Family coverage and censures. affine: 1/2 attempted/planned cases, 540 not_run, 465 ok, 75 unsupported. increment: 1/2 attempted/planned cases, 540 not_run, 465 ok, 75 unsupported. mixed_nonlinear: 2/2 attempted/planned cases, 240 inconclusive, 165 ok, 525 timeout, 150 unsupported. triangular_shear: 1/2 attempted/planned cases, 540 not_run, 465 ok, 75 unsupported. These counts retain all backends and tasks; repetition counts do not measure independent workload diversity.

Table 2: Current qualification, public service boundary. Counts are worker trials, not independent circuits. S: successful service checkpoint; ok: complete qualified trial. S overlaps outcome columns and is not added to total. Every requested status stays in the denominator; U: unsupported; NR: not run; T: timeout; O: OOM; I: inconclusive; K: killed; E: error/other failure.

task	backend	S	ok	total	U	NR	T	O	I	K	E
anf_export	cudd-bdd	60	60	120	0	45	15	0	0	0	0
anf_export	cudd-zdd	60	60	120	0	45	15	0	0	0	0
anf_export	polybori	45	45	120	0	45	30	0	0	0	0
anf_export	qcec	0	0	120	75	45	0	0	0	0	0
anf_export	sat	0	0	120	75	45	0	0	0	0	0
anf_export	sparse	45	45	120	0	45	30	0	0	0	0
anf_export	zdd	45	45	120	0	45	30	0	0	0	0
anf_export	zdd-native	45	45	120	0	45	0	0	30	0	0
anf_export	zdd-native-fused	45	45	120	0	45	0	0	30	0	0
counterexample	cudd-bdd	60	60	120	0	45	15	0	0	0	0
counterexample	cudd-zdd	60	60	120	0	45	15	0	0	0	0
counterexample	polybori	45	45	120	0	45	30	0	0	0	0
counterexample	qcec	0	0	120	75	45	0	0	0	0	0
counterexample	sat	60	60	120	0	45	15	0	0	0	0
counterexample	sparse	45	45	120	0	45	30	0	0	0	0
counterexample	zdd	45	45	120	0	45	30	0	0	0	0
counterexample	zdd-native	45	45	120	0	45	0	0	30	0	0
counterexample	zdd-native-fused	45	45	120	0	45	0	0	30	0	0
equivalence	cudd-bdd	60	60	120	0	45	15	0	0	0	0
equivalence	cudd-zdd	60	60	120	0	45	15	0	0	0	0
equivalence	polybori	45	45	120	0	45	30	0	0	0	0
equivalence	qcec	75	75	120	0	45	0	0	0	0	0
equivalence	sat	45	45	120	0	45	30	0	0	0	0
equivalence	sparse	45	45	120	0	45	30	0	0	0	0
equivalence	zdd	45	45	120	0	45	30	0	0	0	0
equivalence	zdd-native	45	45	120	0	45	0	0	30	0	0
equivalence	zdd-native-fused	45	45	120	0	45	0	0	30	0	0
simulate	cudd-bdd	60	60	120	0	45	15	0	0	0	0
simulate	cudd-zdd	60	60	120	0	45	15	0	0	0	0
simulate	polybori	45	45	120	0	45	30	0	0	0	0
simulate	qcec	0	0	120	75	45	0	0	0	0	0
simulate	sat	0	0	120	75	45	0	0	0	0	0
simulate	sparse	45	45	120	0	45	30	0	0	0	0
simulate	zdd	45	45	120	0	45	30	0	0	0	0
simulate	zdd-native	45	45	120	0	45	0	0	30	0	0
simulate	zdd-native-fused	45	45	120	0	45	0	0	30	0	0

Table 3: Adverse-case public decision and witness services, retained separately. Counts include every requested repeat. T: timeout; I: inconclusive; U: unsupported. All observed I entries here hit the native allocated-manager-node guard, not a final-reachable-DAG size bound or wall-time estimate. QCEC's verdict does not supply the concrete witness required by the last column. These cases lack exhaustive-oracle timing eligibility and never enter the conditional speed ratios.

case	backend	equivalence	counterexample
adverse-nonlinear-14	cudd-bdd	15 ok	15 ok
adverse-nonlinear-14	cudd-zdd	15 ok	15 ok
adverse-nonlinear-14	polybori	15 T	15 T
adverse-nonlinear-14	qcec	15 ok	15 U
adverse-nonlinear-14	sat	15 T	15 ok
adverse-nonlinear-14	sparse	15 T	15 T
adverse-nonlinear-14	zdd	15 T	15 T
adverse-nonlinear-14	zdd-native	15 I	15 I
adverse-nonlinear-14	zdd-native-fused	15 I	15 I
adverse-nonlinear-18	cudd-bdd	15 T	15 T
adverse-nonlinear-18	cudd-zdd	15 T	15 T
adverse-nonlinear-18	polybori	15 T	15 T
adverse-nonlinear-18	qcec	15 ok	15 U
adverse-nonlinear-18	sat	15 T	15 T
adverse-nonlinear-18	sparse	15 T	15 T
adverse-nonlinear-18	zdd	15 T	15 T
adverse-nonlinear-18	zdd-native	15 I	15 I
adverse-nonlinear-18	zdd-native-fused	15 I	15 I

Table 4: Case-level adjudication of the measured manifest. Raw labels and trials are preserved. Increment is deterministic development calibration; new seeds do not create unseen programs when the generator ignores them. Full reasons and the timestamped discovery are in the separate hashed sidecar.

case	adjudicated class	raw split
adverse-nonlinear-14	fresh_adverse_holdout	new_holdout
adverse-nonlinear-18	fresh_adverse_holdout	new_holdout
fresh-affine-10	fresh_stochastic_holdout	new_holdout
fresh-affine-4	fresh_stochastic_holdout	new_holdout
fresh-increment-10	development_calibration	new_holdout
fresh-increment-5	development_calibration	new_holdout
fresh-shear-10	fresh_stochastic_holdout	new_holdout
fresh-shear-5	fresh_stochastic_holdout	new_holdout

Table 5: Current conditional service-time ratio, baseline/shipped-native-fused (> 1: shipped native faster), by family. Each circuit contributes its median paired-repeat ratio; no population generalization. The last column counts distinct common oracle-checked/planned cases within that family, including unrun cases in the planned denominator; repeats are not independent cases.

task	baseline	family	ratio	common/planned
anf_export	cudd-bdd	affine	74.17	1/2
anf_export	cudd-bdd	increment	86.58	1/2
anf_export	cudd-bdd	triangular_shear	82.55	1/2
anf_export	polybori	affine	275.59	1/2
anf_export	polybori	increment	257.99	1/2
anf_export	polybori	triangular_shear	247.55	1/2
anf_export	zdd	affine	1.54	1/2
anf_export	zdd	increment	1.50	1/2
anf_export	zdd	triangular_shear	1.65	1/2
equivalence	cudd-bdd	affine	97.48	1/2
equivalence	cudd-bdd	increment	96.80	1/2
equivalence	cudd-bdd	triangular_shear	76.24	1/2
equivalence	polybori	affine	265.89	1/2
equivalence	polybori	increment	289.58	1/2
equivalence	polybori	triangular_shear	245.58	1/2
equivalence	sat	affine	7.18	1/2
equivalence	sat	increment	5.85	1/2
equivalence	sat	triangular_shear	7.27	1/2
equivalence	zdd	affine	1.63	1/2
equivalence	zdd	increment	1.92	1/2
equivalence	zdd	triangular_shear	2.35	1/2

Table 6: Secondary matched-Python-loop native configuration, not the shipped fused default. Public-boundary baseline/native ratio, equal-family aggregate over common independently oracle-checked cases. The planned denominator includes unrun cases; attempted cases are 5/8. F: represented families; LOFO: leave-one-family-out range. This is a distinct configuration, not a post-hoc best mode.

task	baseline	ratio	F	common/planned	LOFO
anf_export	cudd-bdd	50.96	3	3/8	47.47–56.73
anf_export	polybori	166.28	3	3/8	158.86–176.70
anf_export	zdd	1.00	3	3/8	0.92–1.07
equivalence	cudd-bdd	49.20	3	3/8	46.88–53.27
equivalence	polybori	151.71	3	3/8	148.11–155.05
equivalence	sat	3.59	3	3/8	3.28–4.16
equivalence	zdd	1.05	3	3/8	0.96–1.12

Table 7: Post-audit adjudicated holdout-only sensitivity, excluding deterministic increment calibration without replacing or rerunning cases. Baseline/shipped-native-fused ratio, same estimator and exhaustive-oracle gate as the original mixed study. The planned denominator includes unrun cases; attempted cases are 4/6. Only two small stochastic families qualify for ratios; the adverse family remains in coverage. This is not population inference or a new preregistered primary analysis.

task	baseline	ratio	F	common/planned	LOFO
anf_export	cudd-bdd	78.25	2	2/6	74.17–82.55
anf_export	polybori	261.19	2	2/6	247.55–275.59
anf_export	zdd	1.60	2	2/6	1.54–1.65
equivalence	cudd-bdd	86.21	2	2/6	76.24–97.48
equivalence	polybori	255.54	2	2/6	245.58–265.89
equivalence	sat	7.22	2	2/6	7.18–7.27
equivalence	zdd	1.96	2	2/6	1.63–2.35

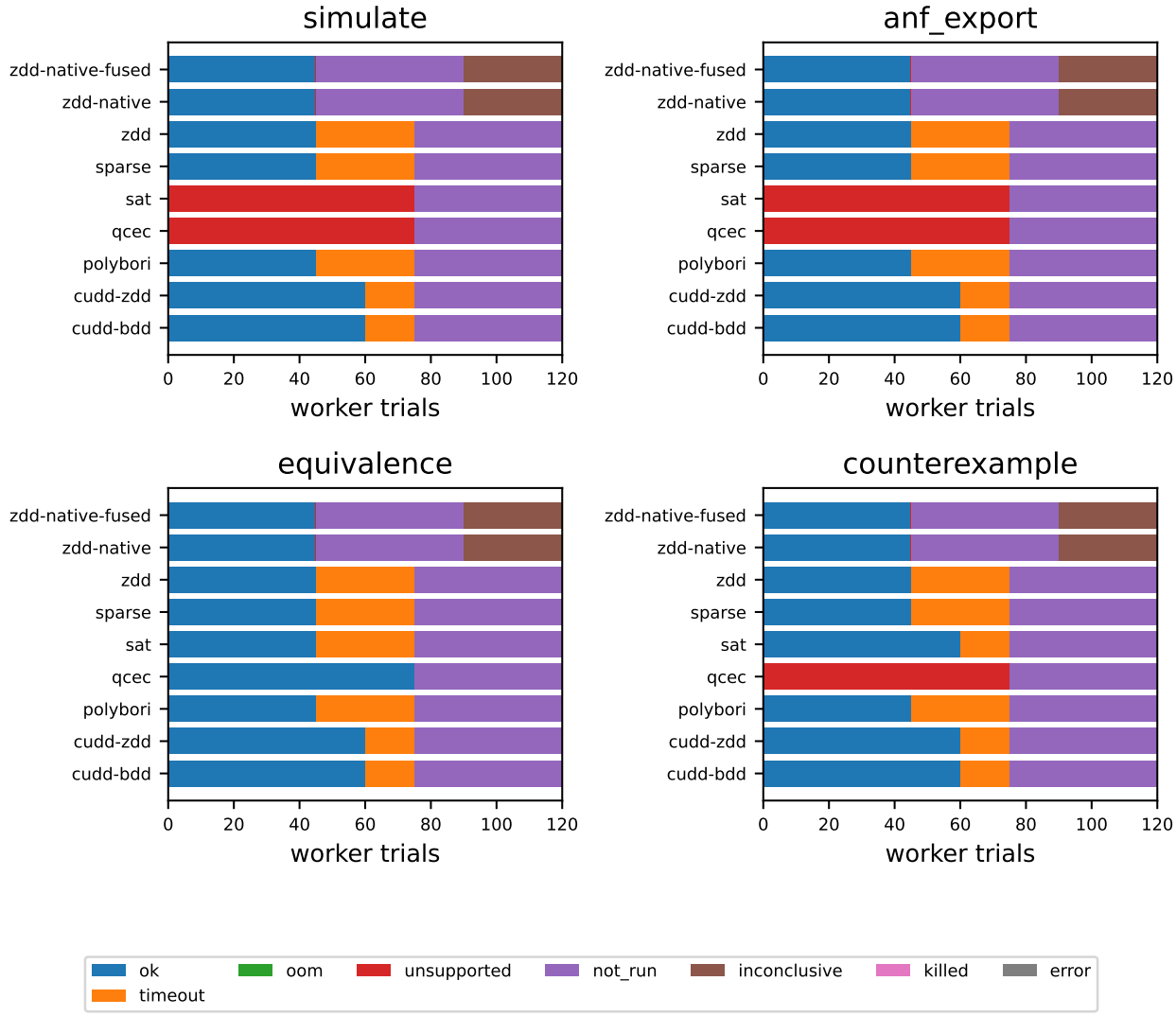


Figure 1: Current public-boundary qualification coverage. Stacks count requested worker trials, including unsupported/unrun work; repeated trials are not independent programs. Task panels remain separate. Ratios computed only on common successes cannot establish this coverage.

anf_export against cudd-bdd: 80.93 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [74.17, 86.58], leave-one-family-out range [78.25, 84.54].

anf_export against polybori: 260.12 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [247.55, 275.59], leave-one-family-out range [252.71, 266.64].

anf_export against zdd: 1.56 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [1.50, 1.65], leave-one-family-out range [1.52, 1.60].

equivalence against cudd-bdd: 89.60 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [76.24, 97.48], leave-one-family-out range [85.91, 97.14].

equivalence against polybori: 266.42 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [245.58, 289.58], leave-one-family-out range [255.54, 277.49].

equivalence against sat: 6.73 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [5.85, 7.27], leave-one-family-out range [6.48, 7.22].

equivalence against zdd: 1.95 on 3/8 common/planned cases and 45 successful repeat pairs, descriptive interval [1.63, 2.35], leave-one-family-out range [1.77, 2.13].

This bounded synthetic qualification establishes only the observed protocol behavior. They do not determine competitiveness on larger compiler-generated circuits, a broad SAT/BDD/PolyBoRi ranking, or the cause of an operation-level effect.

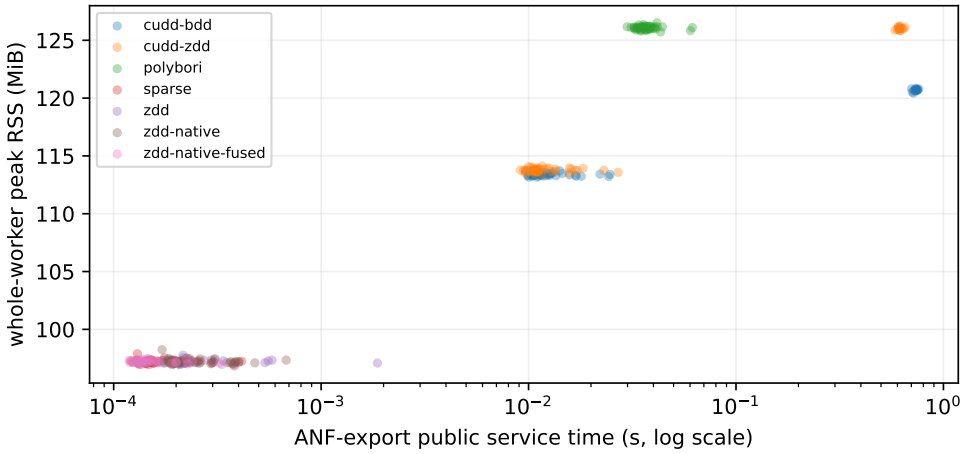


Figure 2: Current ANF-export service cost and whole-worker peak RSS on successful trials. Points are descriptive repeats from different families and sizes, not an isolated causal memory or order ablation. Imports/native heaps contribute to RSS. Failures remain visible in Table 2; they do not disappear from the scientific denominator.

7 Historical exploration and limits of transfer

The October 27 campaigns are preserved at their original sources and native binary `2e3f6791...`, which predates known count-overflow and width-64 import repairs. Their figures concern old drivers/cache regimes and cannot be asserted for today's engine. They found both narrow gains and adverse cases: on the old explored generated corpus the native minimal loop completed 46/90 requested cells, PolyBoRi 46/90, versus 71/90 for dynamic CUDD; these are protocol-specific counts with censoring, not equal-budget solved fractions of independent instances. The old native/PolyBoRi simulation ratio was 3.47 on 15/30 common completed circuits, while the native/dynamic-BDD ratio was 0.69 on 16/30; its interval included one. Factor sorting was adverse, whereas fusion helped after correcting input-construction timing. No one ratio is a statement about current implementation performance.

The old five families had dependent size variants. Binding-cost subtraction was only a sensitivity analysis on calibrated cache-hit operands, not a bound or proof of causality. Source replacement during execution, shared-memory kills and repaired/relabelled cells are listed in `paper/REPRODUCIBILITY.md`; full raw and derived sealed files remain intact. Old tables can be regenerated but are prominently labeled historical. A new campaign does not erase those negative observations or turn the old holdout back into an unseen one.

8 Limits, use and release gates

The measured short corpus qualifies only its cases, interfaces and resource accounting. It does not establish a broad compiler-workload advantage, machine-independent speed, optimal variable order, useful support above 128 variables, or a universal ANF strategy. ANF materialization can be exponential even when a BDD/SAT verdict is cheap. Neither compact ZDD roots nor a wider mask removes that expansion. Full worker RSS is not live-node memory; sanitizer timings are not release timings; precompiled kernels are not public calls. The observed adverse native limit concerns allocated manager nodes, including intermediates and nodes no longer reachable from final roots. It establishes no lower bound on the final reachable DAG or ANF size; collection and different orders might alter the frontier, and their benefit on these particular cases is unmeasured.

The repository also has installed experimental sign/HMR channels, reduced channels, coherent domains, instruments and bounded dyadic/cyclotomic references. Public Qualtran fixtures and small Kraus oracles now exist, unlike the historical prototype description. Those capabilities have their own grammar, input preparation, output observation and coherence obligations; they are not established by the classical propositions here. The separate qualification must reject unjustified extensions and may properly recommend continued experimental status. The separately published public-fixture qualification recommends precisely that status for its two small Qualtran sizes, rather than promotion or a large-multiplier claim. Its immutable data revision is `07f0324c3d9d`; it is not pooled with the classical timings here. Two isolated executions of one oracle provide process independence, not algorithmic diversity. Receipts and disk caches are never proof authority.

The useful outcome is an evidence-based choice: request explicit ANFs only when needed, compare decision engines on decision service, and retain unsupported/resource failures. Scientific release still requires independent

review of the final source, evidence and claims, plus author confirmation of content, metadata, disclosure and licensing. No arXiv submission, release, DOI, external scientific publication or main merge is authorized here.

Artifact identity and reproducibility

Source commit: `49f5c438fe8916d2d2abf0a1dd137c33fd752184`.
Native SHA-256: `2a14a3eac241d2e4de22ebda12880505f8bf0b63b4d07f6a9c9460269b4a8bc4`.
Raw qualification SHA-256: `20810d3735be03677f5245f695dd01683d483b6be8b0494ec6d42615a2d95573`.
Published artifact SHA-256: `797e79198d67d5a6af5fa66b822d1e6936281765bf5916488763f302b6f298c2`.
Case adjudication SHA-256: `2604175d57a3100d420cce3826a6ffdaa850ffa1593d017db27c37a8d9e2b242`.

The raw artifact records the manifest, environment, versions, limits, order and individual trials; `paper/qualification/derived.json` records independent table derivation. `paper/REPRODUCIBILITY.md` gives exact rebuild commands and historical distinctions. The immutable measured source remains its own commit when a later data/paper commit is created; table regeneration never relabels it. Tests and CI attach to exact content.

Author metadata and disclosure

The author name and affiliation above are inherited from the existing draft; this revision adds no author, affiliation, funding or consent claim. The earlier draft reported AI-assisted code, experiments and writing, no funding and no competing interests. The author must reconfirm the final statements before any submission. AI assistance continued during this maintenance and scientific-review revision.

References

[1] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. In *Proceedings 15th International Conference on Quantum Physics and Logic (QPL 2018)*, volume 287 of *Electronic Proceedings in Theoretical Computer Science*, pages 1–21, 2019.

[2] Robert Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification (CAV 2010)*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[3] Michael Brickenstein and Alexander Dreyer. PolyBoRi: A framework for Gröbner-basis computations with Boolean polynomials. *Journal of Symbolic Computation*, 44(9):1326–1345, 2009.

[4] Michael Brickenstein, Alexander Dreyer, Gert-Martin Greuel, Markus Wedler, and Oliver Wienand. New developments in the theory of Groebner bases and applications to formal verification. arXiv:0801.1177 [math.AC], 2008.

[5] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.

[6] Lukas Burgholzer and Robert Wille. Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(9):1810–1824, 2021.

[7] Lukas Burgholzer and Robert Wille. QCEC: A JKQ tool for quantum circuit equivalence checking. *Software Impacts*, 7:100051, 2021.

[8] California Institute of Technology. dd: Binary decision diagrams in pure Python and Cython wrappers of CUDD, Sylvan, and BuDDy. `https://github.com/tulip-control/dd`, 2023. Version 0.6.0, BSD-3-Clause.

[9] Rolf Drechsler, M. Theobald, and Bernd Becker. Fast OFDD-based minimization of fixed polarity Reed–Muller expressions. *IEEE Transactions on Computers*, 45(11):1294–1299, 1996.

[10] Craig Gidney. Halving the cost of quantum addition. *Quantum*, 2:74, 2018.

[11] Wei-Jia Huang, Christophe Chareton, Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Alfons Laarman, and Jingyi Mei. Equivalence checking of quantum circuits via path-sum and weighted model counting. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pages 419–439. Springer, 2026.

[12] Jingyu Ke, Jingyang Li, and Guoqiang Li. Irene: Equivalence checking of hybrid quantum programs via structure-preserving symbolic reduction. arXiv:2609.36065 [cs.PL], 2026.

[13] U. Kebschull, E. Schubert, and W. Rosenstiel. Multilevel logic synthesis based on functional decision diagrams. In *Proceedings of the European Conference on Design Automation (EDAC)*, pages 43–47. IEEE Computer Society Press, 1992.

[14] Vadym Kliuchnikov, Adam Paetznick, and Marcus P. da Silva. Phased outcome-complete simulation. arXiv:2603.24717 [quant-ph], 2026.

[15] Shin-ichi Minato. Zero-suppressed BDDs for set manipulation in combinatorial problems. In *Proceedings of the 30th ACM/IEEE Design Automation Conference (DAC)*, pages 272–277. ACM, 1993.

[16] Shin-ichi Minato. Implicit manipulation of polynomials using zero-suppressed BDDs. In *Proceedings of the European Design and Test Conference (ED&TC)*, pages 449–454, 1995.

[17] Shin-ichi Minato. Zero-suppressed BDDs and their applications. *International Journal on Software Tools for Technology Transfer*, 3(2):156–170, 2001.

[18] Alan Mishchenko, Satrajit Chatterjee, Robert Brayton, and Niklas Eén. Improvements to combinational equivalence checking. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 836–843. IEEE, 2006.

[19] Munich Quantum Toolkit. MQT QCEC: A tool for quantum circuit equivalence checking. `https://github.com/munich-quantum-toolkit/qcec`, 2026. Version 3.11.0, MIT license; test circuits taken from commit 98d6784.

[20] Jérome Ricciardi, Sébastien Bardin, Christophe Chareton, and Benoît Valiron. Quantum circuit equivalence checking: A tractable bridge from unitary to hybrid circuits. arXiv:2511.22523 [quant-ph], 2025.

[21] Fabio Somenzi. *CUDD: CU Decision Diagram Package, Release 3.0.0*. University of Colorado at Boulder, 2015. Mirror: `https://github.com/cuddorg/cudd`, tag cudd-3.0.0.

[22] The BRiAl developers. BRiAl: successor to PolyBoRi. `https://github.com/BRiAl/BRiAl`, 2025. Version 1.2.15, GPL-2.0-or-later.

[23] Robert Wille, Daniel Große, D. Michael Miller, and Rolf Drechsler. Equivalence checking of reversible circuits. In *39th International Symposium on Multiple-Valued Logic (ISMVL)*, pages 324–330, 2009.

[24] Robert Wille, Daniel Große, Lisa Teuber, Gerhard W. Dueck, and Rolf Drechsler. RevLib: An online resource for reversible functions and reversible circuits. In *38th International Symposium on Multiple Valued Logic (ISMVL)*, pages 220–225, 2008.